GPU Implementation and Analysis of Quantized Linear Layers for LLM Inference

Joshua Zhou and Aidan Orias

Motivation and objective

- Linear layers account for a substantial portion of large language model inference computation. During token generation at small batch sizes, loading model weights from GPU memory can limit performance.
- Representing weights with 4-bit integers reduces storage compared with 16-bit floating point, but introduces approximation error and unpacking overhead.
- This project will implement a GPU linear layer using **4-bit weights and floating-point activations**, integrate it into a pretrained language model, and evaluate its accuracy, memory use, and inference performance.
- The project connects Math 221 topics, including matrix approximation, numerical error, blocking, and communication costs, to practical GPU inference.

Implementation plan

- **Reference quantization.** Implement symmetric groupwise quantization in PyTorch, assigning a shared scale to each group of weights. Implement reconstruction and measure its numerical error. Pack two 4-bit weights into each byte, storing scales separately.
- **Custom GPU kernel.** Develop a Triton kernel for $Y = XW$. Load packed weight tiles, unpack and rescale them locally, and multiply by floating-point activations. Avoid reconstructing the entire weight matrix in GPU memory before multiplication. Tune tile sizes and memory layouts, focusing on small batches used during token generation.
- **Model integration.** Replace selected feed-forward linear layers in one pretrained Transformer. Use the existing inference framework for tokenization, attention, caching, and generation.

Numerical accuracy analysis

- Separate **quantization error** from **kernel implementation error**: compare reconstructed weights with the original weights, and compare the custom kernel with reference multiplication using the same reconstructed weights.
- Writing $\hat{W} = W + E$, analyze the exact-arithmetic output perturbation:

$$\|X\hat{W} - XW\|_F \leq \|X\|_2 \|E\|_F.$$

- Compare this bound with measured errors and study how quantization group size affects accuracy. Evaluate model-level quality using next-token loss on held-out text.

Performance evaluation

- **Compare:** the original floating-point linear layer; explicit weight reconstruction followed by library multiplication; the fused quantized kernel; and an existing optimized quantization implementation, where compatible.
- **Measure:** kernel latency across representative matrix dimensions and batch sizes; actual weight storage, including scales and padding; peak GPU memory; and end-to-end generation throughput.
- Use warmup runs and synchronized GPU timing, measuring compilation separately. Use profiling and arithmetic-intensity estimates to explain when reduced memory traffic outweighs unpacking overhead.

Scope and deliverables

- **Core scope:** one GPU architecture, one quantization scheme, one pretrained model architecture, and a small set of group sizes.
- **Deliverables:** a reusable packed 4-bit linear-layer implementation; integration into a pretrained language model; reproducible accuracy and performance experiments; and a report explaining accuracy-memory-performance tradeoffs, including configurations where quantization does not improve speed.